```
# Train.py
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import json

IMAGE_SIZE = (224, 224)
Batch = 32

train_path = "D:\\Internship\\Day55\\fruit_ripeness_dataset\\archive (1)\\dataset\\train"
test_path = "D:\\Internship\\Day55\\fruit_ripeness_dataset\\archive (1)\\dataset\\test"

datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_ds = datagen.flow_from_directory(
    train_path,
    target_size=IMAGE_SIZE,
    batch_size=Batch,
    class_mode='categorical'
)

val_ds = datagen.flow_from_directory(
    test_path,
    target_size=IMAGE_SIZE,
    batch_size=Batch,
    class_mode='categorical'
)

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
    layers.MaxPooling2D(),
```

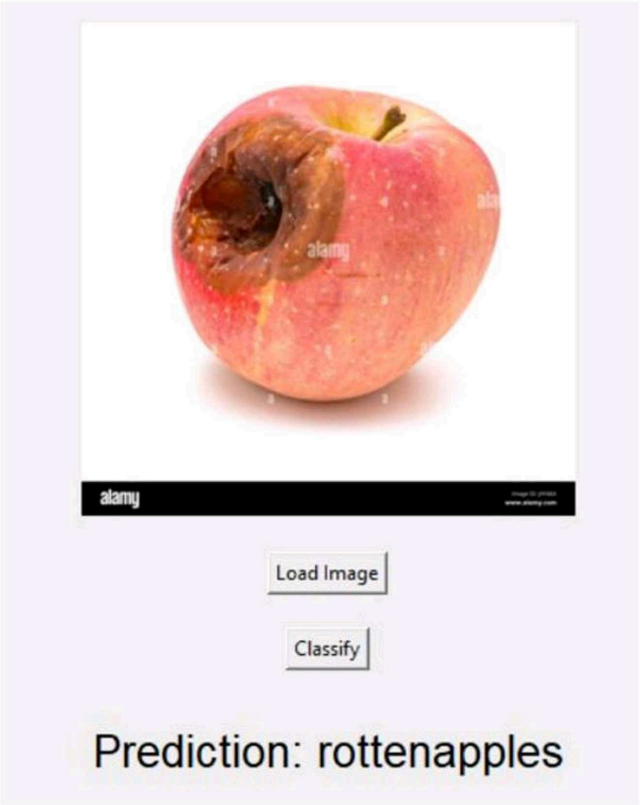
```
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(train_ds.num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

model.fit(train_ds, epochs=10, validation_data=val_ds)

model.save("fruit_ripeness_model.h5")
print("Model saved successfully")

with open("labels.json", "w") as f:
    json.dump(train_ds.class_indices, f)

print("Labels saved successfully")



# Test.py
import tkinter as tk
```

```
from tkinter import filedialog
from PIL import Image, ImageTk
import numpy as np
import tensorflow as tf
import json
from tensorflow.keras.preprocessing import image

model = tf.keras.models.load_model("fruit_ripeness_model.h5")

with open("labels.json", "r") as f:
    class_indices = json.load(f)

labels = list(class_indices.keys())

def predict_image(img_path):
    img = image.load_img(img_path, target_size=(224, 224))
    img = image.img_to_array(img)
    img = img / 255.0
    img = np.expand_dims(img, axis=0)

    result = model.predict(img)
    idx = np.argmax(result)

    return labels[idx]

def open_image():
    global img_path, panel

    img_path = filedialog.askopenfilename(
        filetypes=[("Image files", "*.jpg *.jpeg *.png")]
    )
```

```
    img = Image.open(img_path)
    img = img.resize((300, 300))

    img_tk = ImageTk.PhotoImage(img)

    panel.config(image=img_tk)
    panel.image = img_tk

    result.config(text="")

def classify():
    if img_path:
        label = predict_image(img_path)
        result.config(text=f"Prediction: {label}")

root = tk.Tk()
root.title("Fruit ripeness classifier")
root.geometry("400x500")

panel = tk.Label(root)
panel.pack(pady=10)

btn_load = tk.Button(root, text="Load image", command=open_image)
btn_load.pack(pady=10)
btn_predict = tk.Button(root, text="check fruit and ripeness", command=classify)
btn_predict.pack(pady=10)

result = tk.Label(root, text="", font=("Arial", 20))
result.pack(pady=20)

img_path = ""
root.mainloop()
```

Outputs:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dense_1 (Dense)	(None, 9)	1,161

Total params: 11,170,121 (42.61 MB)
Trainable params: 11,170,121 (42.61 MB)
Non-trainable params: 0 (0.00 B)